Real-Time Face Mask Detection System CL-SSDXcept Implementation and Analysis

Computer Vision and Pattern Recognition - CSE 429 Project Report

Team Members	Marchellino Michel - 120220317 Youssef Yasser Serry - 120220129 Ahmed Elshahat Anwer - 120220270
Project Title	Real-Time Face Mask Detection using CL-SSDXcept
Main Components	CLAHE preprocessing, Caffe SSD face localization, Xception mask classifier
Course	CSE 429: Computer Vision and Pattern Recognition
Instructor	Dr. Ahmed Gomaa
Date	May 2026

Abstract

This report presents the implementation and analysis of a real-time face mask detection system based on the CL-SSDXcept pipeline. The motivation is to build a practical computer vision system that can localize faces from a live webcam stream and classify whether each detected face is wearing a mask. The system uses a pre-trained Caffe SSD detector for face localization, CLAHE preprocessing for illumination normalization, and a fine-tuned Xception model for mask/no-mask classification. The final Xception model achieved 99.1% test accuracy and outperformed MobileNetV2, VGG16, and ResNet50 on the Real-World Masked Face Dataset (RFMD).

Teaser Figure

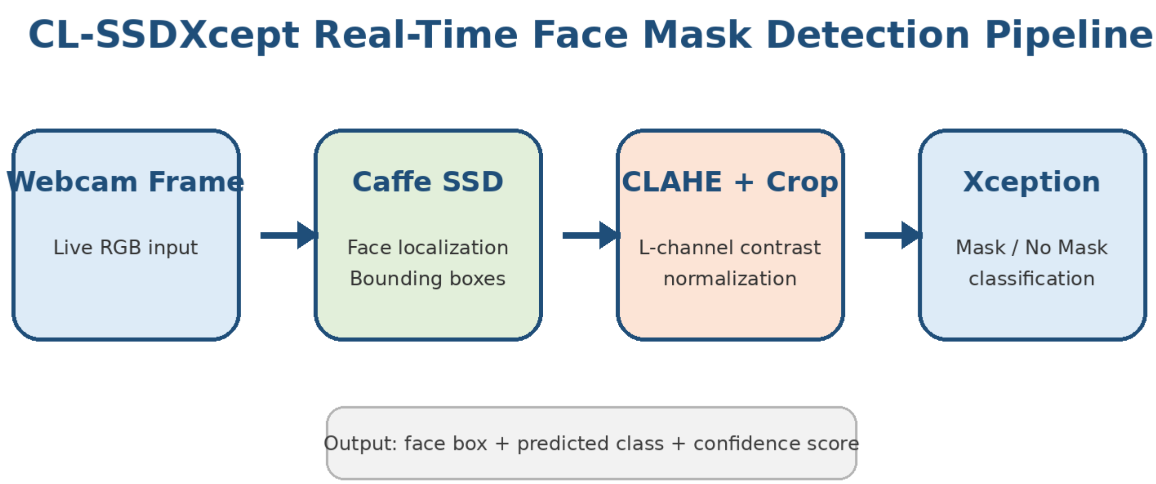

Figure 1. Main CL-SSDXcept pipeline: webcam input, Caffe SSD face localization, CLAHE/cropping, and Xception classification.

1. Introduction

Face mask detection is a practical computer vision application for public health monitoring, access-control systems, campus safety, and intelligent surveillance. A reliable system must solve two sub-problems: face localization and mask classification. Face localization finds the relevant face region, while classification determines whether that face is with mask or without mask.

The project is based on the CL-SSDXcept pipeline. CL refers to contrast enhancement using CLAHE, SSD refers to the Single Shot Detector used for face localization, and Xcept refers to the Xception deep learning model used for mask classification. Compared with using a classifier alone, this two-stage design makes the system more robust because the classifier receives cropped face regions rather than the full image.

2. Project Objectives

- Implement a real-time face localization module using OpenCV DNN and the Caffe SSD face detector.
- Apply CLAHE preprocessing to improve image contrast and reduce lighting variation.
- Train and evaluate a transfer-learning classifier based on Xception.
- Compare Xception with MobileNetV2, VGG16, and ResNet50 to justify the selected model.
- Prepare a clean GitHub repository containing code, model files, result screenshots, and a PDF report.

3. Background

3.1 CL-SSDXcept Pipeline

The CL-SSDXcept pipeline combines image enhancement, object detection, and deep learning classification. CLAHE prepares the image by enhancing contrast, SSD identifies the face region, and Xception performs the final mask/no-mask prediction. This separation of responsibilities helps the system handle real-time webcam input more effectively.

3.2 Caffe SSD Face Detector

SSD, or Single Shot Detector, predicts bounding boxes and confidence scores in a single forward pass. In this project, the pre-trained Caffe SSD face detector uses deploy.prototxt as the network configuration file and res10_300x300_ssd_iter_140000.caffemodel as the detector weights. This detector handles face localization only; the mask classification is handled later by Xception.

3.3 Xception Classifier

Xception uses depthwise separable convolutions, which separate spatial feature extraction from channel-wise feature learning. This architecture is effective for facial features and mask texture patterns. In this project, Xception was fine-tuned rather than kept fully frozen, because fine-tuning improved the final performance significantly.

4. Approach and Methodology

4.1 System Input and Output

Input: live RGB frames from a webcam or prepared dataset images during training. Output: detected face bounding boxes and a mask/no-mask prediction with confidence. During the initial integration stage, the detection script displayed bounding boxes and used a dummy label; the trained classifier can then replace the dummy function.

4.2 Two-Stage Detection and Classification

- Stage 1 - Face Localization: A webcam frame is converted to a 300 x 300 DNN blob and passed to the Caffe SSD detector. The detector returns face bounding boxes and confidence scores.
- Stage 2 - Mask Classification: Each detected face crop is passed to the fine-tuned Xception model for with-mask / without-mask classification.

4.3 CLAHE Preprocessing

CLAHE, or Contrast-Limited Adaptive Histogram Equalization, was applied in the LAB color space. The preprocessing targeted only the L channel, which represents luminance. This normalized lighting and boosted contrast without changing the A and B color channels, preserving color information that may be important for mask appearance and facial texture.

4.4 Detector File Preparation

The Caffe SSD detector requires two standard files:

- deploy.prototxt - defines the Caffe SSD network structure.
- res10_300x300_ssd_iter_140000.caffemodel - contains the trained detector weights.

The detector is loaded using cv2.dnn.readNetFromCaffe(). Each webcam frame is resized to 300 x 300 and converted into a DNN blob using the expected mean values (104.0, 177.0, 123.0).

```
net = cv2.dnn.readNetFromCaffe(PROTOTXT_PATH, MODEL_PATH)
cap = cv2.VideoCapture(1, cv2.CAP_AVFOUNDATION)
```

5. Tools and GitHub Repository Structure

Folder / File	Purpose	Description
src/	Source code	Contains detect_faces.py, preprocess.py, train_gpu.py, and model.py.
models/	Detector files	Contains deploy.prototxt and the Caffe SSD caffemodel file.
saved_models/	Trained model	Contains mask_detector_gpu.h5. Since it is larger than 100 MB, Git LFS or external storage is required.
screenshots/	Visual results	Contains the confusion matrix and training curves.
report/	Submission report	Contains the final PDF report required inside the GitHub repository.
requirements.txt	Dependencies	Lists Python packages needed to run the project.

Final GitHub Repository Structure

```
face_detector/
├── README.md
├── requirements.txt
├── .gitignore
├── src/
│   ├── detect_faces.py
│   ├── preprocess.py
│   ├── train_gpu.py
│   └── model.py
├── models/
│   ├── deploy.prototxt
│   └── res10_300x300_ssd_iter_140000.caffemodel
├── saved_models/
│   └── mask_detector_gpu.h5  (tracked with Git LFS / external if needed)
├── screenshots/
│   ├── confusion_matrix.png
│   └── training_curves.png
```

Figure 2. Final organized repository structure prepared for GitHub submission.

6. Implementation Summary and Obstacles

The detection script loads the Caffe SSD model, opens the webcam stream on macOS, creates a DNN blob for each frame, performs forward inference, and displays bounding boxes around detected faces. During testing, the AVFoundation backend and camera index 1 were required to read frames correctly on the MacBook.

Problem	Cause	Solution
Model file could not be opened	The code used the wrong relative path after reorganizing files.	Moved files into models/ and used stable relative paths.
Camera not authorized	macOS blocked Terminal/Python camera access.	Enabled permission from Privacy & Security > Camera.
Could not read frame	Wrong camera index or backend.	Tested camera indexes and used cv2.CAP_AVFOUNDATION with index 1.
GitHub rejected push	mask_detector_gpu.h5 exceeded the 100 MB GitHub limit.	Removed it from normal Git tracking and prepared Git LFS handling.

7. Experiments and Results

7.1 Experimental Setup

All models were trained on the Real-World Masked Face Dataset (RFMD) for 20 epochs using the Adam optimizer with a learning rate of 1 x 10^-4. Evaluation used validation accuracy, test accuracy, precision, recall, and qualitative inspection through plots and screenshots. A random-guessing classifier would be expected to perform around 50% on a balanced two-class task, so ResNet50 near 50% indicates failure to learn useful class separation under the tested setup.

7.2 Comparative Model Analysis

Xception was compared against MobileNetV2, VGG16, and ResNet50. MobileNetV2 was used as a lightweight baseline, VGG16 represented a classic heavy convolutional network, and ResNet50 tested whether residual skip connections would outperform the depthwise separable convolution design of Xception.

Model	Parameters	Validation Accuracy	Status
Xception (Proposed)	22.9M	99.10%	Optimal
MobileNetV2	3.4M	72.67%	Overfitting
VGG16	138.3M	53.67%	Failed convergence
ResNet50	25.6M	50.00%	Random guessing

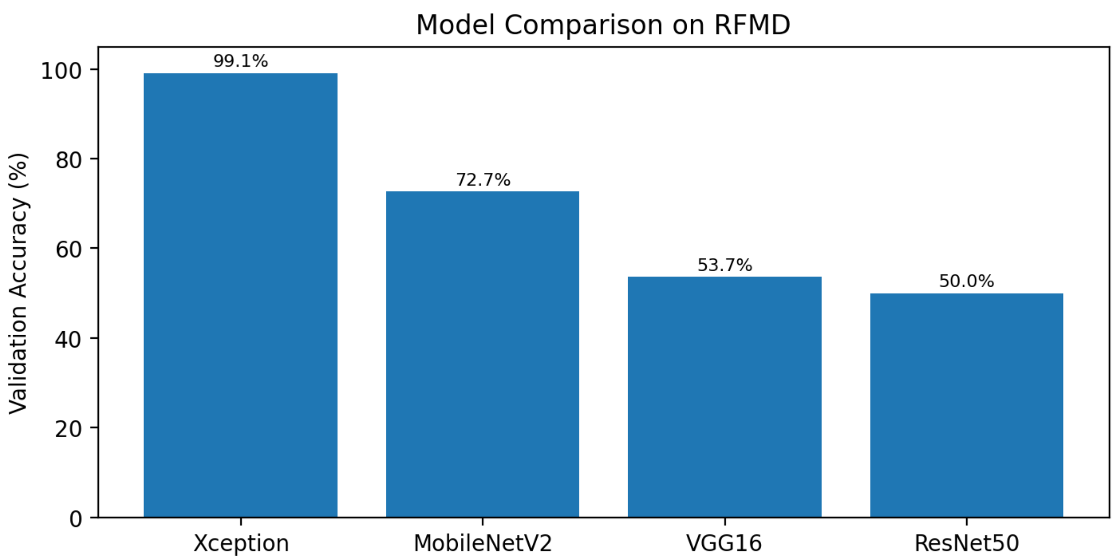

Figure 3. Validation accuracy comparison. Xception achieved the highest validation accuracy among tested models.

7.3 Final Xception Metrics

Metric	Final Xception Result
Test Accuracy	99.1%
Precision / Recall	99.0% for both With Mask and Without Mask classes
Inference Speed	Approximately 697 ms per step on GPU
Frozen Baseline	66% accuracy before fine-tuning
Fine-Tuning Strategy	Top 20 layers unfrozen for improved classification performance

The final results confirm that fine-tuning was essential. The frozen baseline reached only about 66% accuracy, while fine-tuning the top 20 layers improved the model to near-perfect performance. This indicates that pre-trained ImageNet features alone were not sufficient for the specific visual patterns of face mask detection.

8. Qualitative Results

The qualitative evaluation focuses on whether the system behavior matches the expected pipeline visually. The webcam stage should produce a face bounding box around the detected face. The training stage should produce stable learning curves and a confusion matrix that shows most examples classified along the correct diagonal. The repository includes screenshots for training_curves.png and confusion_matrix.png to support this visual evaluation.

Artifact	What It Shows	Expected Interpretation
Webcam detection output	Face bounding box in real time	The SSD detector successfully localizes the face region.
training_curves.png	Accuracy/loss behavior over epochs	Training should improve without severe instability.
confusion_matrix.png	Correct and incorrect predictions by class	High diagonal values indicate strong class separation.

9. Discussion

- Depthwise separable convolutions in Xception were more effective on this dataset than standard convolutional blocks or residual blocks.
- CLAHE improved robustness against lighting variation by enhancing luminance without changing color channels.
- The Caffe SSD detector provided a reliable front-end localization stage for the classification model.
- The complete system depends on both accurate face cropping and a well fine-tuned classifier; poor localization would reduce classification accuracy.

10. Future Work

- Integrate the saved Xception model directly into the real-time detection script and display Mask / No Mask labels above each detected face.
- Use Git LFS for the large mask_detector_gpu.h5 file or provide a Google Drive link inside the README if the repository quota becomes a problem.
- Record a two-minute demo video showing the real-time system, as required for the presentation component.
- Deploy the model with Flask/FastAPI or Streamlit to provide a simple web interface or REST API.
- Test the system under different lighting conditions, backgrounds, mask colors, face angles, and multiple-face scenes.

11. Conclusion

The project implements and analyzes a real-time CL-SSDXcept face mask detection pipeline. The Caffe SSD detector handles face localization, CLAHE improves visual robustness, and the fine-tuned Xception model achieves high classification performance. The comparative analysis demonstrates that Xception is the best-performing architecture among the tested models, achieving 99.1% test accuracy and 99.0% precision/recall for both classes. The final GitHub repository is organized as a self-contained project containing source code, detector files, screenshots, requirements, and the PDF report.

References

1. R. Jayaswal et al., "A face mask detection system: An approach to fight with COVID-19 scenario," Concurrency and Computation: Practice and Experience, 2022.

2. OpenCV Documentation, "Deep Neural Network module," OpenCV 4.x documentation.

3. Real-World Masked Face Dataset (RFMD).

4. Final Project Instructions, CSE 429: Computer Vision and Pattern Recognition, Spring 2026.

5. Project Summary: Real-Time Face Mask Detection, CL-SSDXcept Implementation & Analysis, CSE 429, May 9, 2026.